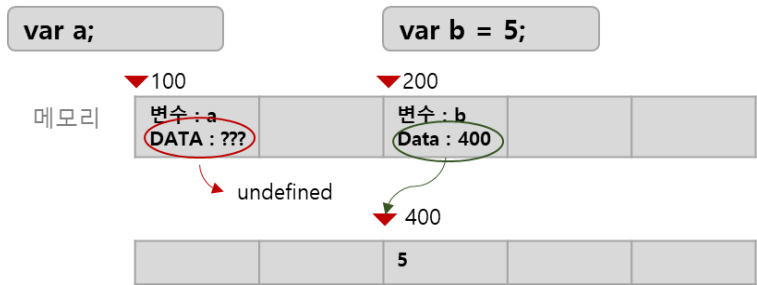


1. 기본

1. 변수

하나의 값을 저장하는 메모리 공간을 식별하기 위해 붙인 이름으로 `var`, `let`, `const` 키워드를 사용하여 선언하고 초기화 한 후 사용합니다. 만약 선언만 하고 초기화 하지 않고 사용하게 되면 `undefined` 값을 가지게 됩니다.



```
var b ;      // 선언
b = 5 ;      // 할당
b = 10 ;     // 재할당
```

자바스크립트는 변수에 대해서 호이스팅이 발생하는데 이것은 [여기를 참조하세요](#).

변수에 저장된 값을 다른 값으로 변경하지 못하는 것을 ****상수(constant)****라 합니다.

2. 표현식 (expression)

값으로 평가될 수 있는 문을 표현식이라 합니다. 여기서 문은 프로그램을 구성하는 기본 단위이자 최소 실행 단위로 프로그래밍을 한다는 것은 순서에 맞게 문을 작성하는 것이고 값은 표현식이 평가(evaluate)되어 생성된 결과 입니다.

```
var vAdd = 10 + 5;
```

3. 리터럴

사람이 이해할 수 있는 문자 또는 약속된 기호를 사용해 값을 생성하는 표기법(notation)

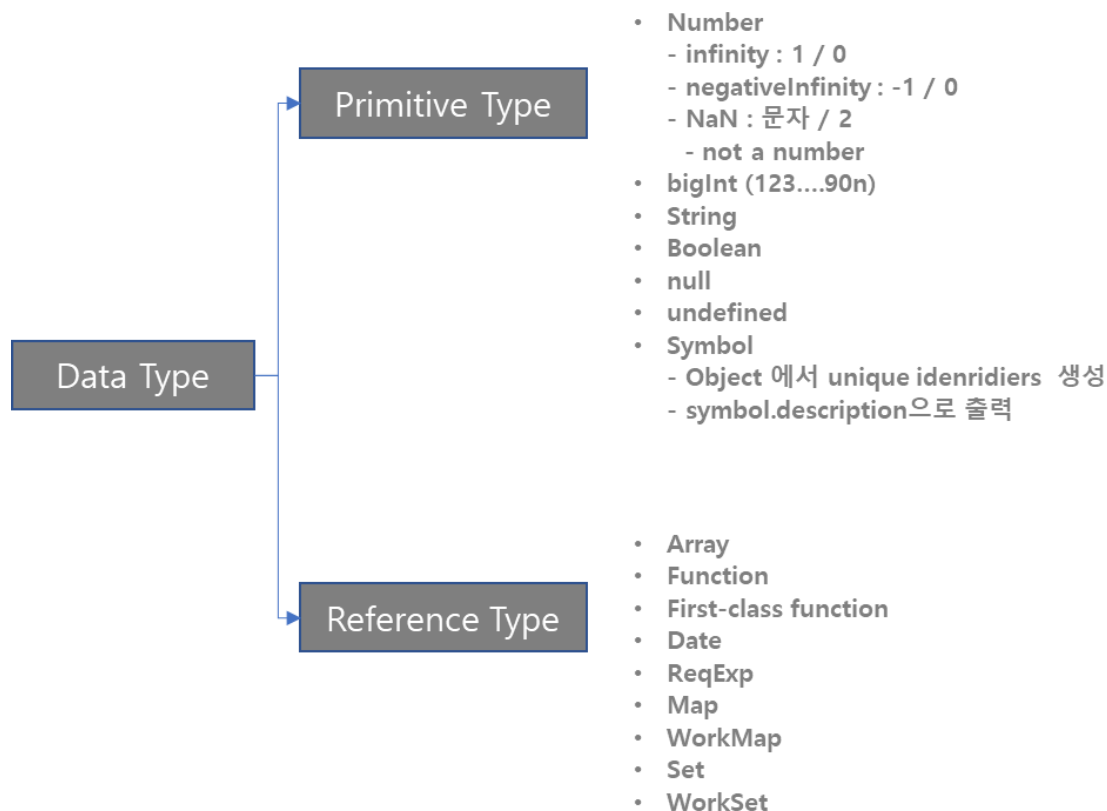
리터럴	예시	비고
정수 리터럴	100	
부동소수점 리터럴	10.5	
2진수 리터럴	0b0100001	0b로 시작
8진수 리터럴	0o101	ES6에서 도입, 0o로 시작
16진수 리터럴	0x41	ES6에서 도입, 0x로 시작

리터럴	예시	비고
문자열 리터럴	'hello' "hello"	
불리언 리터럴	true false	
null 리터럴	null	
undefined 리터럴	undefined	
객체 리터럴	{ name : 'Lee', address: 'Seoul' }	
배열 리터럴	[1, 2, 3]	
함수 리터럴	function() {}	
정규 표현식 리터럴	/[A-Z]+/g	

4. 데이터 타입

값을 저장할 때 필요한 메모리를 확보하고 값을 참조할 때 읽어오기 위한 메모리 공간의 크기를 결정하기 위해 필요합니다.

자바스크립트는 값을 할당하는 시점에 변수의 타입이 결정되는 동적 타입(Dynamic Typing)이며 할당 된 값에 따라서 자유롭게 타입이 변경 됩니다.



4-1. 원시타입

데이터 타입	설명
숫자(number) 타입	- 정수, 실수를 구분하지 않고 하나의 숫자 타입만 가짐 - Infinity : 양의 무한대 -Infinity : 음의 무한대 - NaN: 산술 연산 불가(Not a Number)
bigInt	숫자(Number)의 범위를 벗어나는 값 (숫자 뒤에 n만 붙임)
문자열(string) 타입	- 원시 타입이며, 변경 불가능한 값(Immutable) - 문자 데이터를 문자열로 저장, 작은 따옴표 또는 큰 따옴표로 명시
템플릿 리터럴	- ES6 부터 도입 (템플릿 문자열(Template String)) - 런타임에 일반 문자열로 변환 - 멀티라인 문자열, 표현식 삽입, 태그드 템플릿등 편리한 문자열 처리 기능을 제공
불리언(boolean) 타입	논리적 참과 거짓을 나타내는 true, false뿐이다.
undefined 타입	변수를 선언한 이후 값을 할당하지 않은 변수를 참조하면 undefined가 반환
null 타입	- null타입의 값은 null이 유일하다. (Null, NULL은 해당하지 않는다.) - 의도적으로 변수에 값이 없다는 것을 명시할 때 사용한다. - 누구도 참조하지 않은 메모리 공간에 대해 가비지 콜렉션을 수행 - 함수가 유효한 값을 반환할 수 없는 경우 명시적으로 null을 반환
심벌 타입	- ES6에서 추가된 7번째 타입 - 변경 불가능한 원시 타입의 값으로 다른 값과 중복되지 않는 유일무이한 값 - 다른 원시 값은 리터럴을 통해 생성하지만, 심벌은 Symbol 함수를 통해 생성
객체, 함수, 배열 등	자바스크립트를 이루고 있는 거의 모든 것이 객체

```
var myString = '문자';
var myNumber = 1;
var myBoolean = true;
var myArray = ['a','b','c'];
```

```
var myObject = {"name":"홍길동", "age":"19"};
var myNull = null;

console.log(typeof myString, myString);      // string 문자
console.log(typeof myNumber, myNumber);      // number 1
console.log(typeof myBoolean, myBoolean);    // boolean true

console.log(typeof myArray, myArray, myArray[0]);
// object [ 'a', 'b', 'c' ] a

console.log(typeof myObject, myObject, myObject["name"]);
// object { name: '홍길동', age: '19' } 홍길동

console.log(typeof myObject, myObject, myObject.name);
// object { name: '홍길동', age: '19' } 홍길동

console.log(typeof myNull, myNull); // object null

// 동적 타입
let variable = '문자';
console.log(variable.charAt(0))
console.log(`value : ${variable}, types: ${typeof variable}`);
// value : 문자, type: string

variable = 10;
console.log(`value : ${variable}, types: ${typeof variable}`);
// value : 10, type: number

variable = variable + '20';
console.log(`value : ${variable}, types: ${typeof variable}`);
// value : 1020, type: string

variable = variable / '20';
console.log(`value : ${variable}, types: ${typeof variable}`);
// value : 51, type: number

console.log(variable.charAt(0))
// Uncaught TypeError : variable.charAt is not a function

const LINE = Symbol("line");
const RECTANGLE = Symbol("rectangle");

console.log(LINE.toString());
// > Symbol(line)

console.log(RECTANGLE.toString());
// > Symbol(rectangle)

let shape = LINE;

// 정의한 심볼값만 비교 가능
```

```

if(shape === LINE) {
  console.log("shape === LINE");
} else {
  console.log("shape !== LINE");
}
// > shape === LINE

```

4-2. 객체 타입

데이터 타입	설명
객체, 함수, 배열 등	자바스크립트를 이루고 있는 거의 모든 것이 객체

5. 연산자

5-1. 산술 연산자

+, -, ++, --, *, /, %

5-2. 대입 연산자

=, +=, -=, *=, /=, % =

5-3. 단축 평가

단락 평가란 || or && 연산자를 이용 해서 가장 적합한 정보를 먼저 위치 하여 정보를 확인 하는 것을 의미 합니다. 즉 논리곱 (&&)연산자와 논리합(||)연산자는 논리 연산의 결과를 결정하는 피연산자를 타입 변환하지 않고 그대로 반환 하는 것 입니다. 이때 typeError가 발생 하지 않도록 주의 하여야 합니다.

단축 평가 표현식	평가 결과
true anything	true
false anything	anything
true && anything	anything
false && anything	false

```

const isShotCircuiting = function(param) {
  return param || '기본값';
}

isShotCircuiting('');           // 기본값 ( param : false )
isShotCircuiting(null);         // 기본값 ( param : false )
isShotCircuiting(undefined);   // 기본값 ( param : false )
isShotCircuiting('지정');       // 지정 ( param : true )
isShotCircuiting(true);         // true ( param : true )
isShotCircuiting({});           // {} ( param : true )

```

```
isShotCircuiting([]);           // [] ( param : true )
isShotCircuiting(new Array()); // [] ( param : true )
```

논리곱 예시

```
> let a = 1, b = 1;
< undefined
> a == 1 && b == 1;
< true
> a == 1 && b == 0;
< false
> a == 0 && b == 1;
< false
> a == 0 && b == 0;
< false
> '서울' && b == 1;
< true
> '서울' && b == 0;
< false
> a == 0 && '부산';
< false
> a == 1 && '부산';
< '부산'
> '서울' && '부산';
< '부산'
```

논리합 예

```
> let a = 1, b = 1;
< undefined
> a == 1 || b == 1;
< true
> a == 1 || b == 0;
< true
> a == 0 || b == 1;
< true
> a == 0 || b == 0;
< false
> '서울' || b == 1;
< '서울'
> '서울' || b == 0;
< '서울'
> a == 0 || '부산';
< '부산'
> a == 1 || '부산';
< true
> '서울' || '부산';
< '서울'
```

다음과 같은 경우 활용 할 수 있습니다.

- 객체를 가리키기를 기대하는 변수가 **null** 또는 **undefined**가 아닌지 확인하고 프로퍼티를 참조할 때

코드

```
var element = null;
var value = element.value;
// Uncaught TypeError:
// Cannot read properties of null
// (reading 'value')
```

설명

null을 할당 하면서 오류 발생

코드	설명
<pre>var element = null; var value = element && element.value; // null</pre>	element 값이 null로 false이므로 element 으로 평가됩니다. * true이면 element.value로 평가도됨
함수 매개변수에 기본값을 설정할 때	
<pre>// 단축 평가를 사용한 매개변수의 기본값 설정 function getNameLength(str) { str = str ''; return str.length; } // ES6의 매개변수 기본값 설정 function getNameLength(str = '') { return str.length; }</pre>	
객체를 사용 하는 경우 선언만 하고 항목을 지정 하지 않으면 undefined로 false로 판단 한다	
<pre>const obj = {}; const isShotCircuitingObj = function(obj) { return obj.name '기본값'; }; isShotCircuitingObj(obj); // 기본값 (obj.name -> undefined) const objArray = []; const isShotCircuitingArray = function(obj) { return obj[0] '기본값'; }; isShotCircuitingArray(objArray); // 기본값 (obj.[0] -> undefined) isShotCircuitingArray(obj); // 기본값 (obj.[0] -> undefined)</pre>	
5.4. 옵셔널 체이닝 연산자 ES11에서 도입된 옵셔널 체이닝 연산자 ?.는 좌항의 피연산자가 null 또는 undefined인 경우 undefined를 반환하고, 그렇지 않으면 우항의 프로퍼티 참조 합니다.	
코드	설명

코드	설명
<pre>var element = null; var value = element?.value; // undefined</pre>	element 가 null 이므로 element.value 을 참조하여 undefined 입니다.
<pre>var str = ''; var length = str?.length; console.log(length); // 0</pre>	str 값이 empty string으로 str.length를 참조하여 0을 length에 할당합니다.

5.5. null 병합 연산자

ES11에서 도입된 null 병합 연산자 ??는 좌항의 피연산자가 null 또는 undefined인 경우 우항의 피연산자를 반환하고, 그렇지 않으면 좌항의 피연산자를 반환합니다.

```
var foo = null ?? '기본값';    // 기본값
var foo = '' || '기본값';      // 기본값
var foo = '' ?? '기본값';      // 기본값
```

5.6. 펼침연산자(...)

Object에 있는 항목을 목록으로 변환해주는 연산자로 (...)로 표시 하며 단독으로 사용 할 수 있습니다.

|

```
const arr = [1,2,3];
const sp = [...arr];
var rn = arr === sp ? true : false ;
console.log(sp);    // [1, 2, 3]
console.log(rn);    // false
```

펼침연산자를 통한 데이터 관리

```
const arr1 = [1, 2, 3];
```



```
const arr2 = [4, 5, 6];

// concat()
const concatArr = [...arr1, ...arr2]; // [ 1, 2, 3, 4, 5, 6 ]

// push()
const pushArr = [...arr1, 7]; // [ 1, 2, 3, 7 ]

// splice()
const spliceArr = [...arr1.slice(0, 2)]; //[ 1, 2 ]
```

|| 펼침 연산자를 이용한 데이터 분리 (구조분해 할당) ||

```
const obj = { name: '홍길동',
post: '123-456',
address: '서울 송파' };

const {name, ...address} = obj;
// obj :: { name: '홍길동', post: '123-456', address: '서울 송파' }
// name :: 홍길동
// address :: { post: '123-456', address: '서울 송파' }
```

|

6 조건문/반복

if, switch, while, do/while, for, for/in, ?

```
// 1. if
if (조건) {
  만족;
} else {
  불만족
}

// 2. switch
switch(수식) {
  case 값:
    <실행코드>;
    // break; 인위적으로 삭제 가능하나 코드를 이해하는데 어려움이 있음
  case 값1:
```

```

        <실행코드>
        break;
    case 값2:
        <실행코드>
        break;
    default :
        <값, 값1도 아닌 경우>
}

// 3. while
while (조건) {
    조건이 false 일 때 까지
}

// 3. for
for( 대입문; 조건문; 갱신문) {
    ...
}

// 4. for/in
var myArray = ['a','b','c'];
for (var idx in myArray) {
    console.log(myArray[idx]);
}

// 5. 삼항
조건 ? 만족값 : 불만족값

```

6-1. 객체 비교 하기

- **이중 등호 연산자 (==)** 는 다음과 같은 결과가 나므로 객체 비교 사용시 주의가 필요합니다.
 1. "" == "0" ; // false
 2. 0 == ""; // true
 3. 0 == "0"; // true
 4. false == "false"; // false
 5. false == "0"; // true
 6. false == undefined; // false
 7. false == null; // false
 8. null == undefined; // true
 9. " \t\r\n" == 0 // true
- **삼중 등호 연산자 (===)** 는 삼중 등호는 강제로 타입을 변환하지 않는다는 사실을 제외하면 이중 등호와 동일합니다. 삼중 등호를 사용하면 코드를 좀 더 튼튼하게 만들 수 있고, 비교하는 두 객체의 타입이 다르면 더 좋은 성능을 얻을 수도 있다
 1. "" === "0"; // false
 2. 0 === ""; // false
 3. 0 === "0"; // false
 4. false === "false"; // false
 5. false === "0"; // false
 6. false === undefined; // false

7. `false === null; // false`

8. `null === undefined; // false`

9. `"\t\r\n" === 0; // false`

- **`typeof` 연산자, `instanceof` 연산자, `Object.prototype.toString`**